# push-swap

Sujet du projet – version française

## Objectifs

Push-Swap est un projet très simple qui utilise un **algorithme de tri non comparatif**. Vous disposez d'une liste de valeurs de type `int`, de deux piles (**a** et **b**) et d'un ensemble d'instructions.

Vous devrez écrire **2 programmes** :

- **`push-swap`** : calcule et affiche sur la sortie standard le plus petit programme, écrit dans le langage d'instructions push-swap, qui trie les entiers reçus en arguments.
- **`checker`** : prend des entiers en arguments et lit des instructions sur l'entrée standard. Une fois la lecture terminée, il les exécute et affiche `OK` si les entiers sont triés, sinon `KO`.

Comme indiqué, vous avez deux piles à disposition. Votre but est de trier la pile **a**, qui contient les valeurs entières reçues, par ordre **croissant**, en utilisant les deux piles et un ensemble d'instructions.

### Les instructions disponibles

| | |
|---|---|
| **`pa`** | push a : prend le premier élément au sommet de la pile b et le place au sommet de la pile a |
| **`pb`** | push b : prend le premier élément au sommet de la pile a et le place au sommet de la pile b |
| **`sa`** | swap a : échange les 2 premiers éléments de la pile a |
| **`sb`** | swap b : échange les 2 premiers éléments de la pile b |
| **`ss`** | exécute sa et sb en même temps |
| **`ra`** | rotate a : décale tous les éléments de la pile a d'une position vers le haut ; le premier devient le dernier |
| **`rb`** | rotate b : idem pour la pile b |
| **`rr`** | exécute ra et rb en même temps |
| **`rra`** | reverse rotate a : décale tous les éléments de la pile a d'une position vers le bas ; le dernier devient le premier |
| **`rrb`** | reverse rotate b : idem pour la pile b |
| **`rrr`** | exécute rra et rrb en même temps |

## Exemple

Les piles sont représentées avec le sommet en haut. Initialisation de a et b :

```
Init a et b :
2
1
3
6
8
5
_ _
a b
```

Exécution de `sa` :

```
1
2
3
6
8
5
_ _
a b
```

Exécution de `pb pb pb` :

```
6 3
8 2
5 1
_ _
a b
```

Exécution de `rb` :

```
6 2
8 1
5 3
_ _
a b
```

Exécution de `rra` et `rrb` (équivalent à `rrr`) :

```
5 3
6 2
8 1
_ _
a b
```

Exécution de `pa pa pa` :

```
1
2
3
5
6
8
_ _
a b
```

## Le programme push-swap

Vous devez écrire un programme nommé **push-swap**, qui recevra en argument la pile a sous forme de liste d'entiers. Le premier entier doit se trouver au sommet de la pile.

- Le programme doit afficher la **plus petite liste d'instructions possible** pour trier la pile a, le plus petit nombre se trouvant au sommet.
- Les instructions doivent être séparées par un `\n` et rien d'autre.
- Le but est de trier la pile avec le **minimum d'opérations** possible.
- En cas d'erreur, afficher `Error` suivi d'un `\n` sur la **sortie d'erreur standard**. Sont considérées comme des erreurs : des arguments qui ne sont pas des entiers et/ou des doublons.
- S'il n'y a aucun argument, le programme n'affiche rien (0 instruction).

```
$ ./push-swap "2 1 3 6 5 8"
pb
pb
ra
sa
rrr
pa
pa
$ ./push-swap "0 one 2 3"
Error
$ ./push-swap
$
```

## Le programme checker

Vous devez écrire un programme nommé **checker**, qui recevra en argument la pile a sous forme de liste d'entiers. Le premier argument doit se trouver au sommet de la pile (attention à l'ordre). Si aucun argument n'est donné, checker s'arrête et n'affiche rien.

Checker lit ensuite les instructions sur l'entrée standard, chacune suivie d'un `\n`. Une fois toutes les instructions lues, checker les exécute sur la pile reçue en argument.

- Si, après exécution, la pile a est triée **et** la pile b est vide, checker affiche `OK` suivi d'un `\n` sur la sortie standard.
- Dans tous les autres cas, checker affiche `KO` suivi d'un `\n` sur la sortie standard.
- En cas d'erreur, afficher `Error` suivi d'un `\n` sur la sortie d'erreur standard. Exemples d'erreurs : arguments non entiers, doublons, instruction inexistante et/ou mal formatée.
- S'il n'y a aucun argument, le programme n'affiche rien.

```
$ ./checker "3 2 1 0"
sa
rra
pb
KO
$ echo -e "rra\npb\nsa\n" | ./checker "3 2 one 0"
Error
$ echo -e "rra\npb\nsa\nrra\npa"
rra
pb
sa
rra
pa
$ echo -e "rra\npb\nsa\nrra\npa" | ./checker "3 2 1 0"
OK
$ ./checker
$
```

## Consignes

- Le projet doit être écrit en **Go**.
- Le code doit respecter les **bonnes pratiques**.
- Il est recommandé d'avoir des **fichiers de tests** pour les tests unitaires.
- Le premier exécutable doit s'appeler **`checker`** et le second **`push-swap`**.
- Vous devez être capable de **gérer les erreurs**.

### Paquets autorisés

Seuls les paquets standard de Go sont autorisés.

## Utilisation

```
$ ARG="4 67 3 87 23"; ./push-swap "$ARG" | wc -l
6
$ ARG="4 67 3 87 23"; ./push-swap "$ARG" | ./checker "$ARG"
OK
$
```

Si le programme checker affiche `KO`, cela signifie que votre push-swap a produit une liste d'instructions qui ne trie pas la liste.

## Ce que ce projet vous apprendra

- l'utilisation d'algorithmes de base ;
- l'utilisation d'algorithmes de tri ;
- l'utilisation de piles.

### Aide

https://fr.wikipedia.org/wiki/Algorithme_de_tri (version anglaise : https://en.wikipedia.org/wiki/Sorting_algorithm)